07

PLAN.md: o contrato de execução

Agents são brilhantes e ansiosos: começam a escrever código imediatamente, percebem no meio do caminho que esqueceram o schema do banco, voltam, lembram das variáveis de ambiente — e de repente há 17 arquivos abertos e nenhuma direção clara. O PLAN.md existe para impedir essa cena. Ele fica entre a SPEC (o que construir) e o código (como foi construído), definindo ordem, paralelismo e critérios de conclusão.

A IDEIA CENTRAL DESTE MATERIAL

O PLAN.md é um contrato entre você e os agents: cada um lê e sabe exatamente o que fazer, em que ordem, o que precisa estar pronto antes — e o que entrega ao final. E ele deve ser autocontido: escrito para um agent com zero contexto da conversa de planejamento.

1 Onde ele entra na sequência do Harness

```
SPEC.md                    ← o que construir (material 05)
    ↓
CLAUDE.md / AGENTS.md      ← como trabalhar no projeto (material 05)
    ↓
.claudeignore              ← o que nunca ler (material 06)
    ↓
PLAN.md                    ← VOCÊ ESTÁ AQUI
    ↓
Scaffold                   ← estrutura inicial (material 08)
    ↓
Implementação multi-agents ← execução (material 09)
```

A ordem importa: o PLAN.md vem **antes** do scaffold porque a estrutura precisa refletir a arquitetura que o plano define — não o contrário.

2 A hierarquia: Sprint → Fase → Task

NÍVEL	O QUE É	REGRA-CHAVE
Sprint	Unidade de tempo com um critério de entrega verificável ao final.	O critério cabe em 1 frase testável.
Fase	Grupo de tasks que compartilham o mesmo pré-requisito.	Tem um **gate de teste**: a próxima fase só começa quando os testes da anterior passam.
Task	Menor unidade: um agent, um escopo, um output.	Foco único, autocontida, testável, executável em até ~2h, atribuível a exatamente um agent.

```
Sprint
└── critério de entrega verificável

    Fase 1: sem dependências
    ├── Task 1.1  [Agent A]
    ├── Task 1.2  [Agent B]   ← paralelo
    └── Task 1.3  [Agent C]   ← paralelo

    Fase 2: depende da Fase 1 completa
    ├── Task 2.1  [Agent D]
    └── Task 2.2  [Agent E]   ← paralelo entre si
```

> Tasks que impliquem classes gigantes ou nomes genéricos como `Manager`, `Handler`, `Utils`, `Helper` são sinal de decomposição malfeita — quebre em tasks menores antes de executar.

3 Regras de paralelismo

Tasks da mesma fase podem rodar em paralelo se — e somente se:

- Tocam **módulos ou pastas diferentes**
- Não dependem do **output** uma da outra
- Têm **testes próprios** e independentes

A organização prática é por **ondas (waves)**: a Wave 1 reúne tasks sem nenhuma dependência (paralelismo total); a Wave 2 depende da 1; e assim por diante. Dentro de cada onda, todos os agents rodam simultaneamente.

▸ TESTE RÁPIDO DE INDEPENDÊNCIA

Se você consegue explicar a Task A **sem mencionar** a Task B, elas podem rodar em paralelo. Se não consegue, são sequenciais. O erro mais comum: colocar a task de *integração* na mesma fase das tasks que ela integra — o agent tenta conectar coisas que ainda não existem.

4 Anatomia de uma task

```
#### Task [n.n] — [nome descritivo]
- Agent:  [papel do agent responsável]
- Input:  [o que precisa estar pronto para iniciar]
- Output: [entregável específico, com assinatura ou estrutura]
- Testes críticos:
    - [ ] [comportamento esperado — input válido]
    - [ ] [caso de erro tratado]
```

✗ TASK MAL ESCRITA

"Task 2.1 — Implementar o módulo de processamento. Agent: Backend. Fazer o processamento funcionar. Testes: testar se funciona."

Output vago, agent decide a arquitetura sozinho, teste não verificável, e quem depende desta task não sabe o que vai encontrar.

✓ TASK BEM ESCRITA

Nome descritivo; input explícito; output com **arquivo e assinatura exata** da função; três testes: comportamento com input válido, caso de erro tratado, caso limite coberto.

Três perguntas para validar qualquer task

1. Outro agent consegue usar o output desta task **sem perguntar nada**?
2. O critério de conclusão é verificável por **um comando de terminal**?
3. Os testes estão escritos **antes do código**?

Alguma resposta "não"? Reescreva a task.

5 TDD dentro do PLAN.md

Os testes críticos de cada task são definidos **no plano, antes de qualquer código existir**. O agent escreve o teste primeiro (red), implementa até passar (green), e só então a task está concluída. Cada task tem no mínimo dois testes: o comportamento esperado com input válido e um caso de erro tratado. Cada fase tem um critério de conclusão verificável por comando de terminal.

Isso transforma o PLAN.md numa **especificação executável**: aderência deixa de ser opinião e vira resultado de teste — exatamente o argumento do material 05.

6 Como cada ferramenta separa planejar de executar

FERRAMENTA	IMPLEMENTAÇÃO
Claude Code	Plan Mode nativo (Shift+Tab): somente leitura, gera o plano estruturado para você revisar antes de executar. Dica: use o modelo mais forte no planejamento — raciocínio sobre dependências é onde ele paga o custo.
Cursor	Plan Mode que analisa o codebase, faz perguntas e gera um `.plan.md` editável; há também sistema de Tasks nativo.
Windsurf	Planning Mode: desabilita edições e força o plano explícito antes de qualquer implementação.
Cline	Modos Plan & Act separados — pioneiro do padrão que os demais adotaram.
OpenCode	Sem modo dedicado; o mesmo efeito via instrução: "leia o projeto e gere o PLAN.md — não crie nem edite nenhum outro arquivo".
JetBrains Junie	Fluxo com `plan.md` (abordagem) + `tasks.md` (lista acionável com checkboxes), executado em fases com revisão entre elas.
Copilot Workspace	Planejamento multi-arquivo a partir de issues: planeja, implementa, testa e abre o PR para review.

> O padrão é universal: **planejamento em modo somente leitura → revisão humana → execução**. E o arquivo de plano persiste no repositório como memória entre sessões.

7 Workflow recomendado de ponta a ponta

1. Abrir sessão em **Plan Mode** (somente leitura)
2. Agent lê SPEC.md e CLAUDE.md/AGENTS.md
3. Agent gera o PLAN.md com sprints, fases e tasks
4. **Revisão humana**: editar tasks, ajustar paralelismo
5. Fechar a sessão de planejamento
6. Abrir **nova sessão de execução**, com contexto limpo
7. Executar task por task, com gates de teste entre fases

O investimento antecipado parece lento e não é: é um "waterfall de 15 minutos" que torna todo o desenvolvimento seguinte mais fluido. Quando o código começa, você e o agent sabem exatamente o que estão construindo e por quê.

8 Erros mais comuns

ERRO	CONSEQUÊNCIA
Output vago ("implementar o módulo")	Output de verdade é nome de arquivo, assinatura de função, formato de retorno.
Task de integração junto das tasks base	O agent inventa interfaces ou trava esperando o que não existe.
Paralelo sem verificar dependências	Duas tasks editando o mesmo arquivo = conflito impossível de resolver automaticamente.
Fase sem critério verificável	"Backend pronto" não é critério. Um comando de terminal que retorna sucesso, é.
PLAN.md desatualizado	SPEC mudou ou a task revelou complexidade? Atualize o plano **antes** de continuar — é isso que mantém você no controle.
Planejar e executar na mesma sessão	O raciocínio de planejamento contamina o contexto da execução. Sessão nova, contexto limpo, plano no repositório.

9 Prompt pronto: gerando o PLAN.md

▸ PROMPT PARA GERAR O PLAN.MD

```
Leia os arquivos SPEC.md e CLAUDE.md (ou AGENTS.md) na raiz
do projeto e gere o PLAN.md completo, exatamente nesta estrutura:

# PLAN.md
## Sprint [n] — [critério de conclusão em 1 frase verificável]
### Fase [n] — [nome]
> Dependências: [fases anteriores ou "nenhuma"]
> Paralelismo: [quais tasks desta fase rodam em paralelo]
#### Task [n.n] — [nome]
- Agent: [papel responsável]
- Input: [o que precisa estar pronto]
- Output: [entregável com assinatura ou estrutura]
- Testes críticos:
    - [ ] [comportamento esperado — input válido]
    - [ ] [caso de erro tratado]

Regras obrigatórias:
1. Toda task tem 2+ testes críticos definidos antes do código
2. Nenhum output vago — arquivo, função ou estrutura exata
3. Tasks da mesma fase são genuinamente independentes
4. Nenhuma task de integração antes das tasks base existirem
5. Use os nomes de módulos do SPEC.md — sem nomenclatura nova
6. Cada fase tem critério verificável por comando de terminal

Ao terminar, liste quais tasks rodam em paralelo e quantos
agents serão necessários no total.
```

▸ COLA PARA FIXAR

- PLAN.md é o **contrato de execução**: ordem, paralelismo e critérios — autocontido, para agent com zero contexto.
- Hierarquia: **Sprint** (entrega verificável) → **Fase** (gate de teste) → **Task** (um agent, um escopo, um output).
- Paralelo só quando: módulos diferentes, sem dependência de output, testes próprios.
- Task boa responde três perguntas: output usável sem perguntar? critério por comando? testes antes do código?
- Padrão universal das ferramentas: **planejar em somente leitura → revisar → executar em sessão nova**.
- O plano desatualizado é pior que nenhum plano: **atualize antes de prosseguir**.